Personal Finance Advisor Bot: Project Documentation

Table of Contents

1. Overview
2. Architecture
3. Tech Stack
4. Project Structure
5. Features
6. Application Screenshots
7. User Personas
8. Installation and Setup
9. Configuration
10. Running the Application
11. Public Demo with Ngrok
12. AI Advisory Layer
13. Security and Privacy
14. Testing
15. Deployment Notes
16. Troubleshooting
17. Disclaimer

---

1. Overview

Personal Finance Advisor Bot is a web application that helps individuals and families manage their money. It lets users:

- Record income
- Log and categorize expenses
- Generate tailored monthly budgets
- Monitor emergency fund adequacy
- Receive AI-generated spending insights
- Download financial statements (PDF) and export transactions (CSV)

The application adapts its budgeting strategy to the user's situation, with four built-in personas (salaried professional, college student, freelancer, household manager).

---

2. Architecture

2.1 Core principle: deterministic engine vs. AI

The system separates **calculation** from **explanation**.

Responsibility	Handled by
Budget allocations, category limits	Deterministic Python service layer
Emergency fund calculations	Deterministic Python service layer
Financial health score (0–100)	Deterministic Python service layer
Explanations, commentary, behavioral suggestions	AI provider (Gemini or Mock)

The AI never performs arithmetic. Every number shown to the user comes from the rules engine, so results are reproducible and testable.

2.2 Safety guarantees

1. **Zero PII to AI.** The AI receives only sanitized aggregates (category totals, income totals, goals). Names, emails, account numbers and transaction notes are never sent.
2. **Strict output validation.** Every AI recommendation is schema-validated. If suggested limits exceed spendable income or the format is invalid, the engine falls back to deterministic benchmarks.
3. **Swappable provider.** The AI layer sits behind an `AIProvider` interface, with `GeminiProvider` and `MockProvider` implementations.

2.3 Request flow (conceptual)

```
Browser ──► Flask route ──► Service layer (finance engine) ──► SQLAlchemy ──► DB
                                  │
                                  ├─► Sanitizer (aggregates only)
                                  │        │
                                  │        ▼
                                  │   AIProvider (Gemini / Mock)
                                  │        │
                                  │        ▼
                                  └─◄ Validator (schema + limits, fallback)
```

---

3. Tech Stack

Layer	Technology
Language	Python 3.11+
Web framework	Flask 3.x
ORM	SQLAlchemy
Auth / sessions	Flask-Login
CSRF protection	Flask-WTF
Rate limiting	Flask-Limiter
Database	SQLite (dev/test); PostgreSQL via `DATABASE_URL`
Frontend	Jinja2 templates, custom vanilla CSS design system, Chart.js 4.4, Feather Icons
PDF generation	ReportLab 4.x (pure Python, no native DLLs)
AI SDK	`google-genai`
Public tunneling	Ngrok + ProxyFix middleware
Testing	pytest, pytest-cov

---

4. Project Structure

The layout below is based on paths referenced in the README. Adjust to match your actual tree.

```
Finance advisor/
├── app/                      # Flask application package
│   ├── (models, routes, services, templates, static)
├── scripts/
│   ├── seed_demo_data.py     # Seeds 3 months of data for all personas
│   └── run_with_ngrok.py     # Ngrok-ready server (ProxyFix, prod cookies)
├── tests/
│   ├── test_auth.py
│   ├── test_tracking.py
│   ├── test_finance_engine.py
│   ├── test_budget_validator.py
│   ├── test_api.py
│   └── test_reports.py
├── .env.example
```

```
├── requirements.txt
├── run.py                    # Local development entry point
└── README.md
```

---

5. Features

5.1 Tracking

- Record income entries
- Log expenses with category, amount and optional notes
- Archive categories (rather than delete) to preserve history
- Input validation, including rejection of negative amounts
- Quick-add expense flow designed to complete in under 15 seconds

5.2 Budgeting

- Persona-aware monthly budget generation
- Balanced **50/30/20** model for salaried users
- Strict discretionary caps for students
- Conservative baseline plus **20% tax reserve** for freelancers
- Essentials-first prioritization for households

5.3 Monitoring

- Emergency fund tracking (e.g. a 6-month cushion target for freelancers)
- Allowance **runway / burn-rate alerts** for students
- **Financial health score** from 0 to 100, computed by a deterministic rubric

5.4 Insights

- AI-generated commentary on spending patterns and optimization ideas
- Chart.js visualizations backed by JSON API endpoints

5.5 Reporting

- Downloadable **PDF statements** (ReportLab)
- **CSV export** of transactions

---

6. Application Screenshots

The screenshots below show the dark-themed fintech UI, captured with the seeded **College Student** demo account for October 2026.

6.1 Dashboard



Figure: Dashboard

The dashboard shows the monthly savings goal, the emergency fund cushion (with its target in months), an expenses-by-category donut chart, a budget-vs-actual bar chart, a 6-month income vs. expense trend and an income-by-source breakdown.

6.2 AI Financial Advisor and Insights

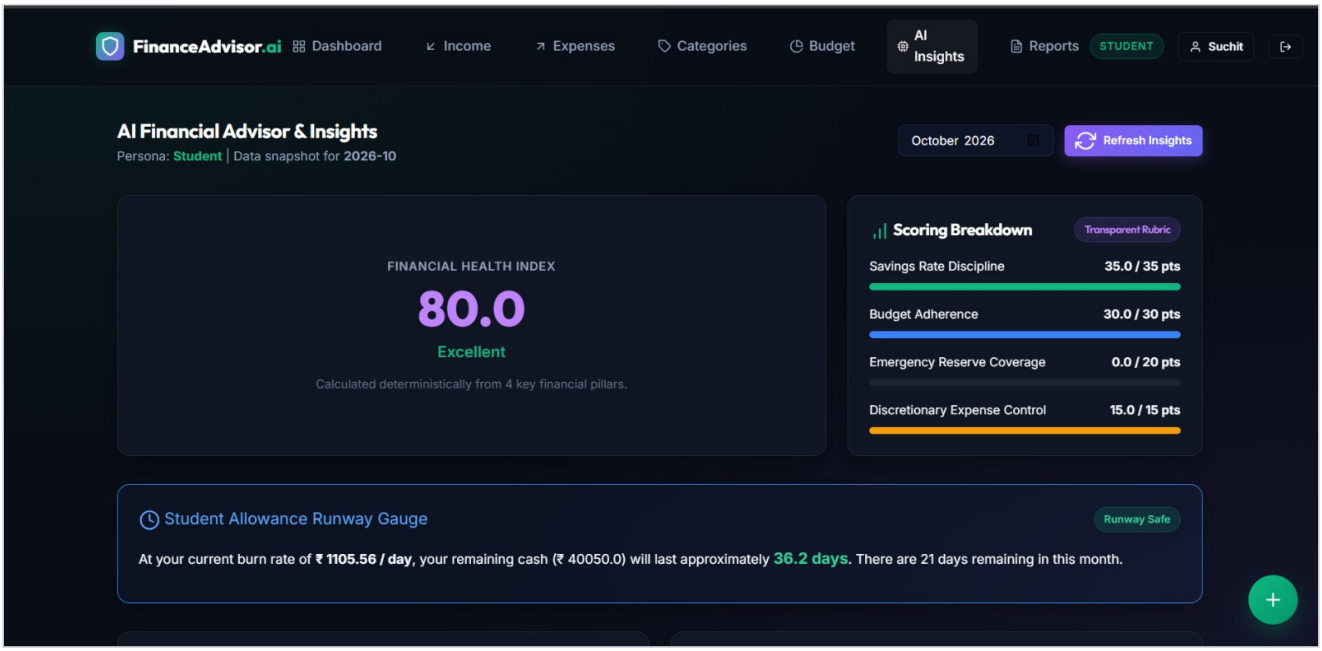


Figure: AI Insights

The insights page shows the 0–100 **Financial Health Index** with its scoring breakdown across four pillars: savings rate discipline (35 pts), budget adherence (30 pts), emergency reserve coverage (20 pts) and discretionary expense control (15 pts). Student accounts also see the **Allowance Runway Gauge**, which projects how many days the remaining cash will last at the current daily burn rate.

6.3 Monthly Budget Management

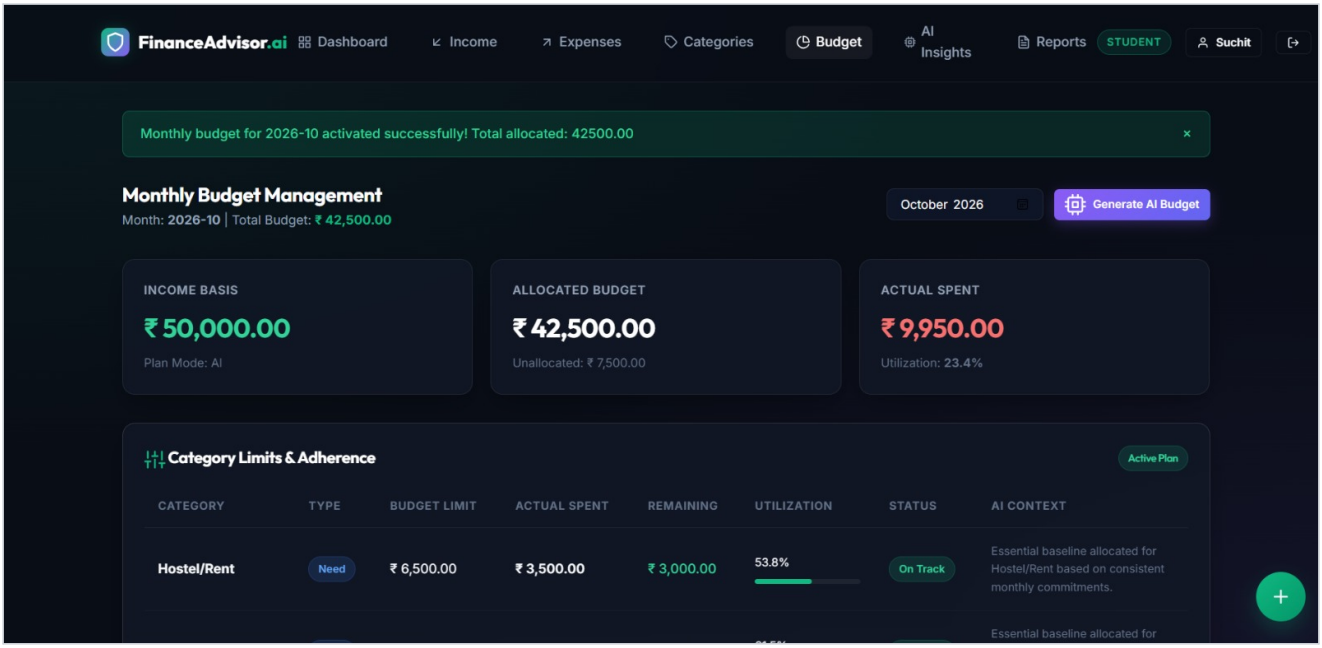


Figure: Budget

The budget page shows the income basis, allocated budget, actual spent and unallocated amount, followed by a per-category table with type (Need/Want), budget limit, actual spent, remaining, utilization bar, status and an AI context note explaining each limit. The **Generate AI Budget** button creates a new plan, which is validated before activation.

---

7. User Personas

All demo accounts use the password `Password123!`.

Persona	Email	Strategy	Default categories
Salaried Professional	`salaried@demo.com`	50/30/20 balanced model	Rent, Groceries, Transport, Utilities, EMI/Loans, Dining out, Entertainment, Shopping
College Student	`student@demo.com`	Limited allowance, runway burn-rate alert, strict discretionary caps	Hostel/Rent, Food, Books/Education, Travel, Subscriptions, Entertainment
Freelancer / Consultant	`freelancer@demo.com`	Variable income, 20% tax reserve, conservative baseline, 6-month emergency fund	Rent, Food, Software/Tools, Project expenses, Taxes, Travel
Household Manager	`household@demo.com`	Shared multi-income budget, dependents tracking, essentials first	Groceries, Utilities, Education, Healthcare, Rent/EMI, Household help

> These accounts are for demonstration only. Do not reuse the demo password anywhere real.

---

8. Installation and Setup

Prerequisites

- Python 3.11 or newer
- Git

Steps

1. Clone the repository

```
git clone <repo-url>
```

```
cd "Finance advisor"
```

2. Create a virtual environment

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure the environment

```
cp .env.example .env
```

Then edit `.env` (see Configuration).

5. Seed demo data

```
python scripts/seed_demo_data.py
```

This populates three months of realistic transactions for all four personas.

---

9. Configuration

Settings are read from `.env`.

Variable	Purpose	Example
`SECRET_KEY`	Flask session/CSRF signing key. Must be strong and unique.	long random string
`DATABASE_URL`	Database connection string. Defaults to local SQLite if unset.	`postgresql://user:pass@host/db`
`AI_PROVIDER`	`mock` (default fallback) or `gemini`	`gemini`
`GEMINI_API_KEY`	Required when `AI_PROVIDER=gemini`	your key

Check `.env.example` for the full, authoritative list of variables.

Using Gemini instead of the mock provider:

```
AI_PROVIDER=gemini
GEMINI_API_KEY=your_actual_gemini_api_key_here
```

Generate a strong secret key with:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

---

10. Running the Application

```
python run.py
```

Open http://localhost:5000 and sign in with any demo account, or register a new one.

---

11. Public Demo with Ngrok

Use this to show the app to external stakeholders.

Terminal 1: start the Ngrok-ready server

```
python scripts/run_with_ngrok.py
```

This enables ProxyFix (correct HTTPS header forwarding) and production cookie policies.

Terminal 2: start the tunnel

```
ngrok http 5000
```

Security rules for public demos

- Ngrok URLs are reachable by anyone on the internet.
- Use **only** the seeded demo accounts.
- **Never** enter real bank details, PAN/Aadhaar numbers or other personal information.
- Make sure `SECRET_KEY` is strong and unique.
- Shut the tunnel down when the demo ends.

---

12. AI Advisory Layer

12.1 Providers

Provider	Use
`MockProvider`	Offline/local default; deterministic canned advice, no API key needed
`GeminiProvider`	Live advice via Google Gemini (`google-genai`)

New providers (other LLMs) can be added by implementing the `AIProvider` interface and registering it under a new `AI_PROVIDER` value.

12.2 What the AI sees

Only sanitized aggregates, for example:

- Income totals
- Per-category spending totals
- User-defined goals

It never sees names, emails, account numbers, or free-text notes.

12.3 Validation and fallback

Each AI response passes through the budget validator:

1. **Schema check**: the response must match the expected structure.
2. **Limit check**: suggested category limits must not exceed spendable income (total ≤ income).
3. **Retry**: on failure, the engine may retry.
4. **Fallback**: if validation still fails, deterministic benchmark limits are used instead.

This means a bad or hallucinated AI response cannot produce an invalid budget.

---

13. Security and Privacy

Control	Implementation
Password storage	Hashed (never stored in plaintext)
Session management	Flask-Login
CSRF protection	Flask-WTF
Rate limiting	Flask-Limiter
Data isolation	Per-user scoping to prevent IDOR (insecure direct object reference)
PII to AI	None; aggregates only
Proxy/HTTPS	ProxyFix and secure cookie policy for tunneled demos

---

14. Testing

Run the full suite with coverage:

```
# Windows
.venv/Scripts/pytest -v --cov=app tests/

# macOS / Linux
.venv/bin/pytest -v --cov=app tests/
```

Test file	Covers
`tests/test_auth.py`	Registration, password hashing, sessions, CSRF, IDOR isolation
`tests/test_tracking.py`	Income/expense CRUD, negative amount validation, category archiving
`tests/test_finance_engine.py`	50/30/20 adaptations, freelancer 20% tax reserve, student runway burn-rate, 0–100 health score rubric
`tests/test_budget_validator.py`	Limit validation (total ≤ income), retry logic, fallback safety
`tests/test_api.py`	Chart.js JSON endpoints, <15s quick-add expense flow
`tests/test_reports.py`	ReportLab PDF generation, CSV transaction export

---

15. Deployment Notes

The README targets local development and Ngrok demos. For a real deployment, consider:

- Use PostgreSQL via `DATABASE_URL` instead of SQLite.
- Run behind a production WSGI server (e.g. Gunicorn) and a reverse proxy with TLS.
- Set a strong, unique `SECRET_KEY` and keep `.env` out of version control.
- Use a persistent rate-limit storage backend (such as Redis) for Flask-Limiter when running multiple workers.
- Do not seed or expose demo accounts in production.

---

16. Troubleshooting

Problem	Likely cause / fix
`ModuleNotFoundError`	Virtual environment not activated, or dependencies not installed. Re-run `pip install -r requirements.txt`.
Empty dashboards	Demo data not seeded. Run `python scripts/seed_demo_data.py`.
AI advice looks generic	`AI_PROVIDER` is still `mock`. Switch to `gemini` and set `GEMINI_API_KEY`.
Gemini errors or fallback budgets every time	Invalid API key, quota exceeded, or responses failing validation; the engine is falling back safely.
CSRF errors on forms	Missing/expired session, or `SECRET_KEY` changed mid-session. Reload and retry.
Login loops behind Ngrok	Use `scripts/run_with_ngrok.py` so ProxyFix and cookie settings are applied.
PDF generation fails	Confirm `reportlab` installed correctly (`pip show reportlab`).

---

17. Disclaimer

The Personal Finance Advisor Bot is an educational and informational tool for budgeting discipline and spending clarity. It does **not** provide certified financial, investment, legal, or tax advice. All numerical figures, budget limits and health scores are computed deterministically by system rules.